EX12 - Vertical Motion with Functions - Splitting Responsibilities

Beginner Step-by-Step Guide

This guide follows the real file `ex12_vertical_motion_functions.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Reorganize EX08 into separate functions for validation, simulation, one Euler step, analytical reference and error calculation.

Problem variables / elements

Name	Meaning
validate_inputs	validates inputs
simulate_vertical_motion	runs the simulation
euler_step	computes one step
analytical_solution	computes reference
calculate_errors	compares results

Equation or logic on paper

```
Euler + formulas analiticas de EX08
```

Manual check

The first step remains h=5.0, v=49.019, t=0.1. Refactoring should not change the physics.

2. What we already know and what is new

Already seen:

- functions
- while
- Euler
- multiple returns

New in this exercise:

- separation of responsibilities
- function calling another
- return True/False
- local variables
- orchestration

3. New Python concepts

1. Responsibility

Syntax: `def euler_step(...):`

Each function aims to do one clear task. euler_step only computes the next state.

2. Function calls function

Syntax: `h, v, t = euler_step(...)`

simulate_vertical_motion reuses euler_step instead of duplicating its equations.

3. Validation Boolean

Syntax: `return True / return False`

validate_inputs communicates a decision to the outer code.

4. Code step by step

Step 1. Separate validation

```
def validate_inputs(h0, v0, t0, g, dt):
    ...
    return True
```

If a check fails it immediately returns False; if all pass it returns True.

Step 2. Create euler_step

```
def euler_step(h, v, t, g, dt):
    h_new = h + v * dt
    v_new = v - g * dt
    t_new = t + dt
    return h_new, v_new, t_new
```

The function has clear inputs and outputs and can be tested independently.

Step 3. Reuse inside simulation

```
while h >= 0:
    h, v, t = euler_step(h, v, t, g, dt)
```

The call returns three values and updates the state.

Step 4. Separate reference and errors

```
analytical_solution(...)
calculate_errors(...)
```

Simulation, reference calculation and error calculation are different responsibilities.

Step 5. Orchestrate

```
if validate_inputs(h0, v0, t0, g, dt):
    max_h, v_f, t_f, i = simulate_vertical_motion(...)
    ...
```

The main block decides the order: validate, simulate, compare and display.

5. Execution flow

```
validate_inputs
 -> simulate_vertical_motion
    -> euler_step repeatedly
 -> analytical_solution
 -> calculate_errors
 -> print
```

6. Expected result / check

Results should match EX08 for the same inputs: ~129.92 m, 10.30 s and -51.04 m/s with dt=0.1.

7. Common beginner mistakes

- Duplicating the same equations across functions.
- Using globals when they should be parameters.
- Confusing a function name with a function call.
- Changing return order.

8. If you remember only five things

- Functions split responsibilities.
- One function can call another.
- True/False can control flow.
- return connects functions through data.
- Refactoring should not change physical results.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.